Programming 1

What is programming?
An organized list of instructions

Eg: Event agenda, Recipe

What is a Computer Program?

Computer program, detailed plan or procedure (organized list of Instructions) for solving a problem with a computer.

ALGORITHM

What is an algorithm?

A step-by-step process to solve a problem.

A sequence of activities to be processed for getting desired output from a given input.

Basic Structure of an algorithm

- Sequence
- Selection/ Branching
- Repetition/ Iteration/ Looping

Example:

Problem 2: Write an algorithm to convert temperature Fahrenheit to Celsius

Inputs: Temperature in Fahrenheit.

Process: C = 5/9 * (F-32)

Output: Temperature in Celsius.

Algorithm:

Step1: Start

Step 2: Read Temperature in Fahrenheit F

Step 3: C = 5/9*(F - 32)

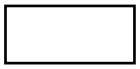
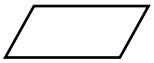
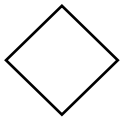
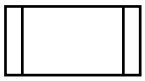
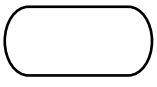
Step 4: Print Temperature in Celsius: C

Step5: End

FLOWCHART

What is a flowchart?

The flowchart is a diagram that visually presents the flow of operations/data through processing systems.

Symbol	Name	Function
	Process	Indicates any type of internal operation inside the Processor or Memory
	Input/ Output	Used for any Input / Output (I/O) operation. Indicates that the computer is to obtain data or output results
	Decision	Used to ask a question that can be answered in a binary format (Yes/No, True/False)
	Connector	Allows the flowchart to be drawn without intersecting lines or without a reverse flow.
	Predefined Process	Used to invoke a subroutine or an Interrupt program.
	Terminal	Indicates the starting or ending of the program, process, or interrupt program
	Flow Lines	Shows the direction of the flow

PYTHON

History of Python

It was a Dutch programmer, Guido Van Rossum, who wrote Python as a hobby programming project back in the late 1980s. Since then, it has grown to become one of the most polished languages in the computing world.

What is Python?

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics.

Python Keywords

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Operator Keywords

Math	**Python**
AND	and
OR	or
NOT	not

Data types in Python

Python Numbers

- Integers- **int**
- Floating point numbers- **float**
- Complex numbers- **complex**

In addition, under sequence category - **list, tuple, range**

We can use the **type()** function to know the type of variable or value belongs to.

Using built-in functions **int(), float(),** and **complex()** we can convert between types. These functions can even convert from strings.

Python Strings

String is a sequence of characters. We can use single quotes or double quotes to represent strings. Multi-line strings can be denoted using triple quotes, ‘’’ or “””.

Eg: ‘Kamal Perera’

Python Booleans

Booleans represent one of two values: True or False

Python Variables

A variable is a named location used to store data in memory.

Eg: Number=10

Number→Variable

10→Assigned value

- Variable names should have a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or underscore (_).
- Never use special symbols like !, @, #, $, %, etc.
- Don't start the name with a digit.
- Use CamelCase notation to declare a variable. It describes a compound word that uses capital letters to delimit the word parts. It resembles the humps on a camel's back

Eg: MyName, MyAge, MyAddress

Python Statement

Eg: A=1 is an assignment statement

Python Indention

Eg: for i in range(1,11):

```
    print(i)
    if i == 5:
        break
```

Python Comment

Eg: # This is a comment.

```
# This statement is to print 'Hello'
```

Python Input and Output

Python Input Using input() function

Eg: num = input('Enter a number ')

Python Output Using print() function

Eg: print('Hello World!!!')

Problem 1: Write an algorithm to read two numbers and find their sum.

Inputs: First number num1, Second number num2.

Process: Sum = num 1 + num2

Output: Sum (sum of the two numbers.)

Algorithm:

Step1: Start

Step2: Read\input the first num1.

Step3: Read\input the second num2.

Step4: Sum = num1+num2

Step5: Write/Output Sum

Step6: End

Python Program

```
Num1 = int(input('Enter first number'))
Num2 = int(input('Enter second number'))
Sum = Num1 + Num2
print(Sum)
```

Python Operators

Relational Operators

Operator	Meaning	Example	Result
==	Equals	1+1 == 2	True
!=	Does not equal	3.2 != 2.5	True
<	Less than	10 < 5	False
>	Greater than	10 > 5	True
<=	Less than or equal	126<=100	False
>=	Greater than or equal	5.0>=5.0	True

Logical Operators

Operator	example	Result
and	9 != 6 and 2<3	True
or	2 == 3 or -1<3	True
not	not 7>0	false

Conditional Statements

if statement:

Executes a group of statements only if a certain condition is true. Otherwise, the statements are skipped.

Syntax:

```
    if condition:
```

Eg:

```
weight = float(input("How many kilos does your suitcase weight? "))
if (weight > 50.0):
  print("There is a $25 for luggage that heavy..")
Print("Thank you for your business…")
```

If..else statement:

Executes one block of statements if a certain condition is True, and the second block of statements if it is False.

Syntax:

```
if condition:
        statements
else:
        statements
```

Eg:

```
GPA = 1.4
If GP > 2.0):
        print "Welcome to IIT University! “
else:
        print "Your application is denied."
```

Operator precedence rule in Python

Operator	Meaning
()	Parentheses
**	Exponent
*,/,//,%	Multiplication, Division, Floor division, Modulus
+,-	Addition, Subtraction
==,!=,>,>=,<,<=, not, in. not in	Comparisons, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR

What are Lists?

List is a data type in Python that is an ordered sequence of elements where elements are mutable.

Mutable means that you can change the items inside, while the ordered sequence is in reference to the index location.

Eg: nums = [0, 10, 4, 20, 17]

List Methods

Append: list.append(x)

If L is the name of an existing list, we can append an item to the end of the list using method Append. Returns the list with newly added elements.

e.g if L=[4,12,3] L.append(5) will give L=[4,12,3,5]

Insert: list.insert(i,x)

Insert an item at a given position. The first argument is the index of the element before which to insert. Returns the list with newly added elements.

e.g if L = [4,12,3]
L.insert(0,15) will give L = [15, 4, 12, 3]
L.insert(2,24) will give L = [15, 4, 24, 12, 3]

Remove: list.remove(x)

Remove the first item from the list whose value is x. It is an error if there is no such item. Returns the balance list after removing the first x.

e.g if L = [4,12,3,12] L.remove(12) will give L = [4, 3, 12]

Pop: list.pop(x)

Remove the item at the given position in the list, and return it. If the **NO** index is specified, a.pop() removes and returns the last item in the list.

e.g if L=[4,12,3,34,56] L.pop(3) will give L=[4,12,3,56] and return 34.

Clear: list.clear(x)

Remove all items from the list and returns an empty list.

Length: len(L)

Returns the length of the list (number of items in the list).

e.g if L=[4,12,3,34,56] len(L) will return 6.

list.**extend**(iterable)

Extend the list by appending all the items from the iterable. Equivalent to a[len(a):] = iterable.

list**.index**(x, start, end)

Return the zero-based index in the list of the first item whose value is x. Raises a ValueError if there is no such item. The optional arguments start and end are interpreted as in the slice notation and are used to limit the search to a particular subsequence of the list. The returned index is computed relative to the beginning of the full sequence rather than the start argument.

list**.count**(x)

Return the number of times x appears in the list.

list**.sort**(key=None, reverse=False)0

Sort the items of the list in place (the arguments can be used for sort customization, see sorted() for their explanation).

list.**reverse**()

Reverse the elements of the list in place.

list.**copy**()

Return a shallow copy of the list. Equivalent to a[:]

Python range() function

Syntax:
range (start, stop,[step])

eg:
range(1,5) – gives the sequence 1,2,3,4
range(5) – gives 0,1,2,3,4
range(1,9,2) – gives 1,3,5,7,9

Repetition

Loops in Python

1. While
2. For

The While Loop

This type of loop can be used if it's **not known** in advance how many times the loop will repeat.
Syntax:

(Simple condition)

while **(Boolean expression):**
    body

(Compound condition)

while **(Boolean expression)** Boolean operator **(Boolean expression):**
    body

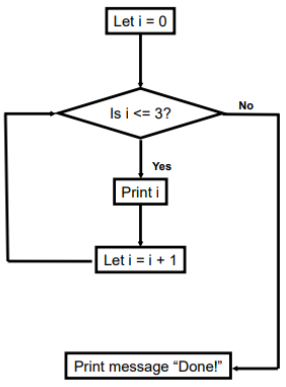
The While Loop (1)

```
i = 0
while (i <= 3):
    print("i =", i)
    i = i + 1

print("Done!")
```

Execution :

Variable i	0	1	2	3	4
is i <= 3?	Yes	Yes	Yes	Yes	No
Print i	0	1	2	3	
i = i + 1	1	2	3	4	



Countdown with while Loop

Eg:

```
i = 3
while (i > 0):
        print("i =", i)
        i = i – 1
print("Done!")
```

The For Loop

In Python, a for-loop is used to step through a sequence.

Syntax:

for<name of loop control> in <something that can be iterated>:
        body

while: while_increment.py

```
i = 0
while (i <= 10):
        print("i =", i)
        i = i + 1
print("Done!")
```

For: for_increment.py

```
for i in range (0, 10):
        print("i =", i)
print("Done!")
```

Counting Down with A For Loop

```
i = 0
total = 0
for i in range (3, 0, -1):
        total = total + i
        print("i = ", i, "\t total = ", total)
print("Done!")
```

For Loop with else

A for loop can have an optional else block as well. The else part is executed if the items in the sequence are used for loop exhausts.

Eg:

```
digits = [0, 1, 5]
for i in digits:
        print(i)
else:
        print("No items left.")
```

Loop Increments Need Not Be Limited To One

while: while_increment5.py

```
i = 0
while (i <= 100):
    print("i =", i)
    i = i + 5
print("Done!")
```

For: for_increment5.py

```
for i in range (0, 105, 5):
    print("i =", i)
print("Done!")
```

```
i = 0
i = 5
i = 10
i = 15
i = 20
i = 25
i = 30
i = 35
i = 40
i = 45
i = 50
i = 55
i = 60
i = 65
i = 70
i = 75
i = 80
i = 85
i = 90
i = 95
i = 100
Done!
```

Sentinel Controlled Loops

- The stopping condition for the loop occurs when the 'sentinel' value is reached.
- **Program name**: sum.py

```
total = 0
temp = 0
while(temp >= 0):
    temp = input ("Enter a non-negative integer (negative to end
      series): ")
    temp = int(temp)
    if (temp >= 0):
        total = total + temp

print("Sum total of the series:", total)
```

```
Enter a positive integer (negative to end series):1
Enter a positive integer (negative to end series):2
Enter a positive integer (negative to end series):3
Enter a positive integer (negative to end series):-1
Sum total of the series: 6
```

Q: What if the user just entered a single negative number?

James Tam

Nested Loops

- One loop executes inside of another loop(s).
- **Example structure**:

```
Outer loop (runs n times)

    Inner loop (runs m times)

        Body of inner loop (runs n x m times)
```

Testing Loops

- Make sure that the loop executes the proper number of times.
- Test conditions:
  1) Loop does not run
  2) Loop runs exactly once
  3) Loop runs exactly 'n' times

Practice Exercise

- Program name: `nested.py`

```
i = 1
while (i <= 2):

    j = 1

    while (j <= 3):

        print("i = ", i, " j = ", j)
        j = j + 1

    i = i + 1

print("Done!")
```

Testing Loops: An Example

Program name: `testing.py`

```
sum = 0
i = 1
last = 0

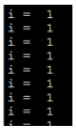
last = int(input("Enter the last number in the sequence to sum : "))
while (i <= last):
    sum = sum + i
    print("i = ", i)
    i = i + 1

print("sum =", sum)
```

Infinite Loops

- Infinite loops never end (the stopping condition is never m
- They can be caused by logical errors:
  - The loop control is never updated (Example 1 – below).
  - The updating of the loop control never brings it closer to the stoppin condition (Example 2 – next slide).
- **Example 1**: `infinite1.py`

```
i = 1
while (i <= 10):
     print("i = ", i)
i = i + 1
```



To stop a program with an infinite loop in Unix simultaneously press the <ctrl> and the <c> keys

Programming 2

Python Data Types

Text Type

str

Numeric Types

int, float, complex

Sequence Types

list, tuple, range

Mapping Type

dict

Set Types

set, frozenset

Boolean Type

bool

Binary Types

bytes, bytearray, memoryview

Escape Character

\' for a single quote

\\ for backslash

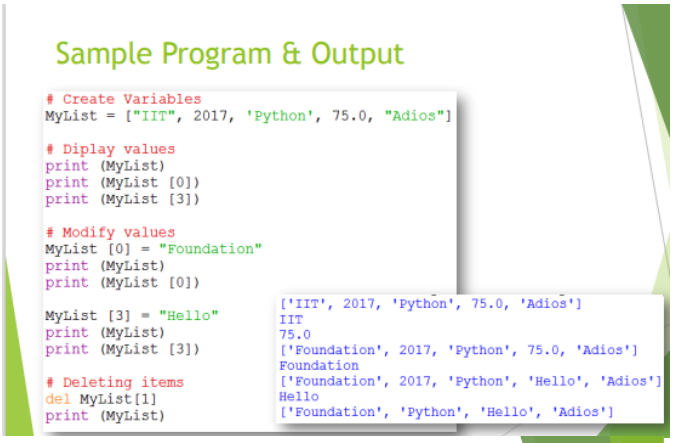
\n for a new line

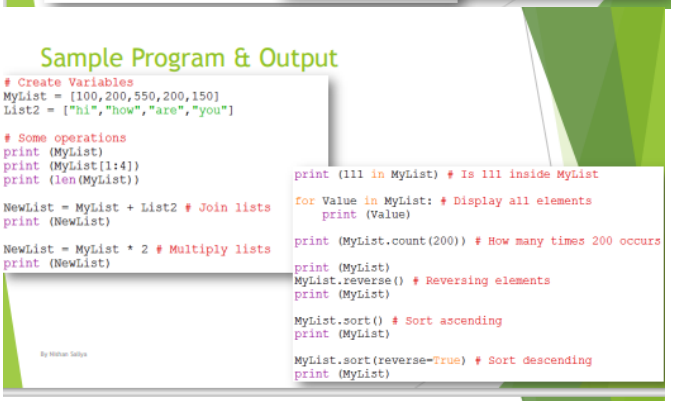
\r for carriage return

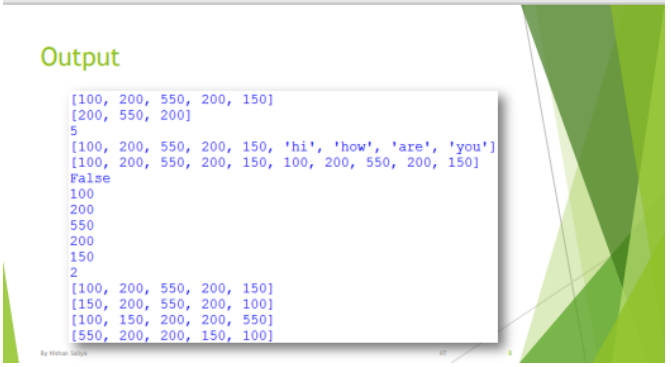
\t for tab

\b for backspace

Python Lists





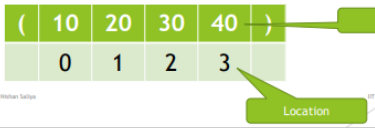


Tuples In Python

A Tuple

Take a look at the below sample codes for Tuples

- `MyTuple1 = ()`
- `MyTuple2 = (50,)`
- `MyTuple3 = (10, 20, 30, 40)`
- `MyTuple4 = (2017, "Python", "Cut-off", 50.00)`



Sample Tuple Program

```
# Create tuples
MyTuple1 = () # Empty tuple
MyTuple2 = (50,) # Single value tuple
MyTuple3 = (10, 20, 30, 40)
MyTuple4 = (2017, "Python", "Cut-off", 50.00)

# Display Tuples
print (MyTuple1)
print (MyTuple2)
print (MyTuple3)
print (MyTuple4)

# Display specific values (using slicer)
print (MyTuple2[0])
print (MyTuple3[2])
print (MyTuple4[1:3])
```

Update Tuple Elements

```
# Create tuples
MyTuple = (10, 20, 30, 40)

# Display Tuple
print (MyTuple)

# Update Tuple
MyTuple [2] = 60

# Display Tuple
print (MyTuple)
```

Create New Tuple

```
# Create tuples
Set1 = (1,2,3,4)
Set2 = (10,20,30,40)

print (Set1)
print (Set2)

# Join tuples
New = Set1 + Set2
print (New)

# Delete Tuples
del Set2

print (Set1)
print (Set2) # Possible ???
```

Simple Tuple Operations

```
# Create tuples
MyTuple = (10,20,30,40)

print (MyTuple)
print (len(MyTuple)) # Size
print (MyTuple * 2) # Two times
print (30 in MyTuple) # Is value there

# Dipslay with a loop
for Value in MyTuple:
    print (Value)
```

Convert List to a Tuple

```
# Create variables
MyList = [100, 200, 300, 400]
MyTuple = 0
print (MyList)
print (MyTuple)

# List -> Tuple
MyTuple = tuple(MyList)
print (MyTuple)
```

Convert Tuple to a List

```
# Create variables
MyTuple = ("Oh", "My", "God", "Python", "getting", "difficult")
MyList = 0

print (MyTuple)
print (MyList)

# Tuple -> List
MyList = list(MyTuple)
print (MyList)

# Updated to Son
MyList [2] = "Jesus"
print (MyList)
```

Strings in Python

Concatenate

```
# Declare variables
Company = "Informatics"
Short = 'IIT'

# Display values
print (Company)
print (Short)

# Concatenate (join)
FullName = Company + Short
print (FullName)

FullName = Company + " " + Short
print (FullName)
```

Display Specific Values

```
# Declare variables
Name1 = "Nishan"
Name2 = 'Saliya'

# Display values
print (Name1)
print (Name2)

# Specific values
print (Name1[4])
print (Name1[1:4])
```

N	I	S	H	A	N
0	1	2	3	4	5

N	I	S	H	A	N
0	1	2	3	4	5

Other String Operations

```
# Declare variables
Name = "IIT"

# Display values
print (Name)

# Other operations
print (Name * 2) # Repeat strings
print ("I" in Name) # Is "I" inside Name
print ("H" not in Name) # Is "H" not in Name

print (len(Name)) # Length of Name
print (Name.capitalize())
print (Name.upper())
print (Name.lower())
print (Name.center(7,"*"))
print (Name.index("I",0,3)) # Location of "I"
print (Name.isnumeric())
print ("55".isnumeric())
```

Functions in Python

Types of Functions

- Simple Functions
- Functions accept values
- Function return values
- Functions accept parameters and return values

Simple Function

```
# Simple fucntion
def MyFun():
    "This line is for nothing"
    print ("I'm from a function")
    return

# ------------------------- Starting main body ---

print ("Main program body")
# Calling the fucntion
MyFun()
print ("End of program")
```

: Simple Function [Explain]

```
# Simple fucntion
def MyFun():
    "This line is for nothing"
    print ("I'm from a function")
    return

# ------------------------- Starting main body ---

print ("Main program body")
# Calling the fucntion
MyFun()
print ("End of program")
```

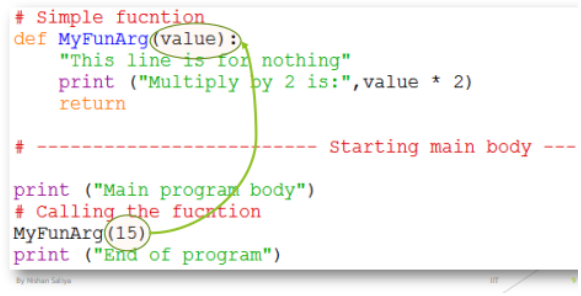
Function accepts values

```
# Simple fucntion
def MyFunArg(value):
    "This line is for nothing"
    print ("Multiply by 2 is:",value * 2)
    return

# ------------------------- Starting main body ---

print ("Main program body")
# Calling the fucntion
MyFunArg(15)
print ("End of program")
```

Function Accepting Arguments [Explain]



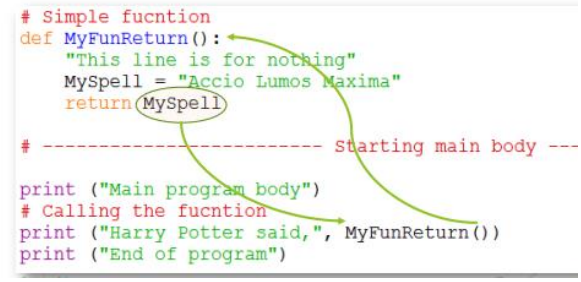
Functions returning values

```
# Simple fucntion
def MyFunReturn():
    "This line is for nothing"
    MySpell = "Accio Lumos Maxima"
    return MySpell

# ------------------------- Starting main body ---

print ("Main program body")
# Calling the fucntion
print ("Harry Potter said,", MyFunReturn())
print ("End of program")
```

Function Returning Values [Explain]



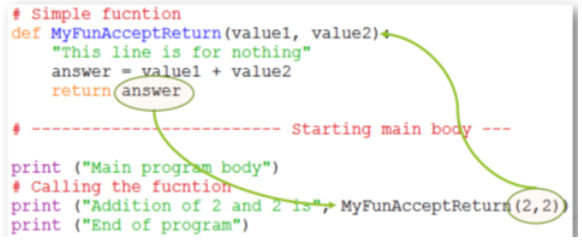
Function accept and return values

```
# Simple fucntion
def MyFunAcceptReturn(value1, value2):
    "This line is for nothing"
    answer = value1 + value2
    return answer

# ------------------------- Starting main body ---

print ("Main program body")
# Calling the fucntion
print ("Addition of 2 and 2 is", MyFunAcceptReturn(2,2))
print ("End of program")
```

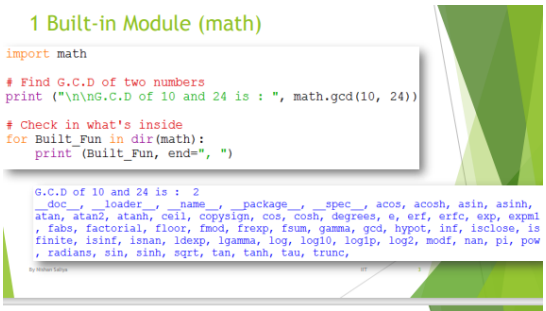
Function Accept and Return Values [Explain]

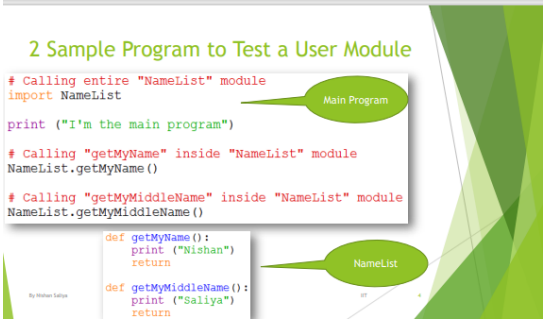


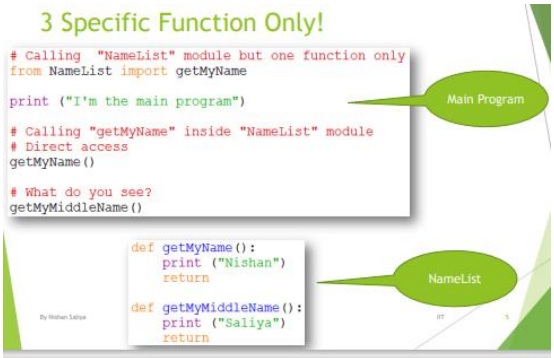
Modules

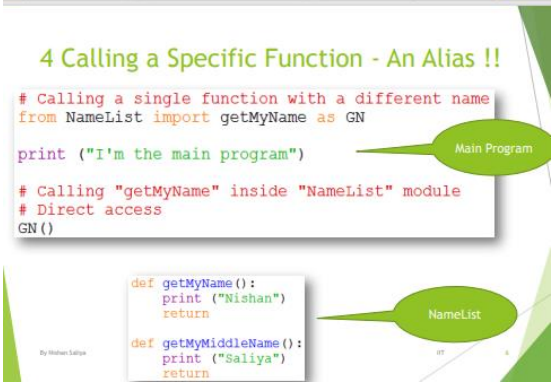
What is a module?

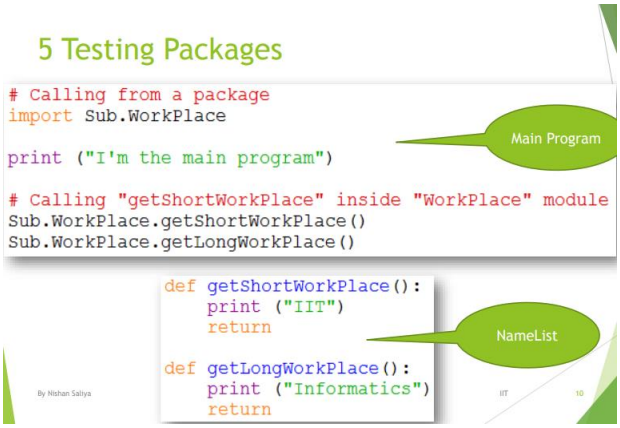
- Python module is simply a file.
- This file helps to organize your Python code.
- A module may contain functions.











File Handling

File Access in Python

Files can be text files or binary files

Files can be opened with some options such as

- Read only
- Write only
- Append only
- Read and/or write

Some Text file open modes

Modes	details
r	Reading only. The file pointer at the BOF
r+	For reading and writing. The file pointer at the BOF
w	Writing only. If a file exists, it'll be overwritten; If it doesn't a new file will be created for writing.
w+	For writing and reading. If a file exists, it'll be overwritten; If it doesn't a new file will be created for reading and writing.
a	Appending. The file pointer is at EOF if the file exists. If the file does not exist, it creates a new file for writing
a+	For appending and reading. The file pointer is at EOF if the file exists. If the file does not exist, it creates a new file for reading and writing.

Sample Program to Read a File

```
# Creating variables
fo = 0

# You have to have a file called
# MyText.txt with some text in the same folder

# Open a text file with
fo = open("MyText.txt", "r")
print ("File name : ", fo.name)
print ("Which mode? : ", fo.mode)
print ("What is in it?\n==============")
print(fo.read())

# Accessing files in sub folder
fo2 = open("Sub\MyTextSub2.txt", "r")
print ("What is in it?\n==============")
print(fo2.read())
```

You need some sample text files to test this!

Sample Program to Write to a File

```
# Creating variables
fo = 0

# You may have a file called
# MyText.txt with some text in the same folder

# Open a text file with write only mode
fo = open("MyText.txt", "w")
fo.write("Hello")
fo.close()
```

Reading a File - Line by Line

```
# Creating variables
fo = 0

# Open a text file with
fo = open("MyTextLines.txt", "r")
with fo:
    count = 1
    for line in fo:
        # Reading line by line
        print ("Line", count , "-", line, end="")
        count += 1
```

Reading a File - Character by Character

```
# Creating a variable
fo = 0

# Opening a text file
fo = open ("ReadThis.txt","r")

for x in range (39):
    print (fo.read(1), end="")
```

```
Contrary to popular belief
Lorem Ipsum
```

Seek a File - Part 1

```
# Creating variables
fo = 0

# Opening a text file read-binary
fo = open ("ReadThis.txt","r")
print ("My location : ", fo.tell())

# Read 10 characters only
print (fo.read(10))
print ("My location : ", fo.tell())

print (fo.read(3))
print ("My location : ", fo.tell())

# Seeking to 14th Location (absolute position)
fo.seek(14,0)
print ("My location : ", fo.tell())
print (fo.read(4))
print ("My location : ", fo.tell())
```

Seek a File - Part 2

```
fo.seek(21,0)
print ("My location : ", fo.tell())
print (fo.read(2))
print ("My location : ", fo.tell())

fo.seek(30,0)
print ("My location : ", fo.tell())
print (fo.read(4))
print ("My location : ", fo.tell())

fo.seek(0,2) # go to end of file
print ("My location : ", fo.tell())

fo.close()
```

How Seeking is Working

o	n	e		t	w	o		t	h	r	e	e		f	o	u	r
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

19 & 20

f	i	v	e		s	i	x		s	e	v	e	n		e	i	g	h	t
21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40

41 & 42